overview-26042026

Overview — counter.c & Execuções 22-04-2026

[26-04-2026][Marcus Vinicius Freitas Margarites Filho]

O que o programa faz

`counter.c` é um benchmark de concorrência que mede o custo de diferentes mecanismos de sincronização POSIX tentando incrementar um contador até TARGET = 1 000 000 000. Quatro experimentos, dois modelos de execução (threads / processos), dois níveis de sync (nenhum / sincronizado):

Exp	Workers	Sincronização	Primitivas POSIX centrais
T1	Threads	Nenhuma	`pthread_create`, `pthread_join`
T2	Threads	Mutex	+ `pthread_mutex_lock/unlock`
P1	Processos	Nenhuma	`fork`, `wait`, `shmget/shmat/shmdt/shmctl`
P2	Processos	Semáforo nomeado	+ `sem_open/wait/post/close/unlink`

O binário aceita `<EXP> <N>` (N ∈ {2, 4, 8}) e roda um experimento por invocação. O timing é feito externamente via `time`.

Avaliação do uso da API POSIX

T1 — Threads sem sincronização

Correto:

- `pthread_create` com atributos NULL (padrão POSIX joinable) e argumento por ponteiro.
- `pthread_join` para cada thread antes de retornar — sem leak de threads detached.

Problemático:

- `counter_t1` é uma global acessada por N threads sem nenhuma sincronização. Isso constitui
  data race no sentido de C11 (`_Atomic` ausente) e no sentido POSIX (acesso concorrente sem
  lock). É comportamento indefinido formalmente.
- Ausência de `volatile`: o compilador tem liberdade de cachear `counter_t1` num registrador
  dentro do `while`. Em `-O2` ou superior isso pode causar loop infinito (lê uma vez, nunca re-lê). Funcionou em `-O0`/`-O1` mas não é portável.

T2 — Threads com `pthread_mutex`

Correto:

- `PTHREAD_MUTEX_INITIALIZER` para mutex estático — inicialização correta sem `pthread_mutex_init`.
- Padrão lock → check → increment → unlock é o padrão canônico de seção crítica.
- Unlock no branch de saída (`break`) antes de retornar — sem deadlock.

Questionável:

- `pthread_mutex_destroy` nunca é chamado. Para mutex com storage estático em Linux é inofensivo, mas POSIX requer destroy para liberar recursos associados.
- Granularidade de lock por incremento individual gera contenção extrema (intencional para o
  experimento, mas vale documentar como anti-padrão de produção).

P1 — Processos com memória compartilhada, sem sync

Correto:

- `shmget(IPC_PRIVATE, sizeof(long), IPC_CREAT | 0666)` — semântica IPC_PRIVATE garante
  segmento exclusivo, sem colisão de chave.
- `shmat` / `shmdt` nos filhos antes de `exit(0)` — limpeza correta.
- `shmctl(shmid, IPC_RMID, NULL)` no pai após `wait` — remove o segmento do kernel.
- `wait(NULL)` em loop de N iterações — coleta todos os filhos, sem zumbis.

Problemático:

- Mesmo problema de `volatile` do T1, mas menos agudo: entre processos separados o compilador
  não pode cachear a variável em registrador cruzando o fork, então na prática é seguro.

Ainda
assim, acessos sem sincronização via memória compartilhada produzem corridas reais (vide overshoots nos resultados).

P2 — Processos com semáforo nomeado

Correto:

- `sem_unlink(SEM_NAME)` antes de `sem_open(O_CREAT|O_EXCL)` — defesa contra semáforo órfão
de execução anterior.
- Padrão `sem_wait` → check → increment → `sem_post` é correto; o `sem_post` no branch de
saída evita deadlock dos outros filhos.
- `sem_close` + `sem_unlink` no pai após `wait` — remoção completa do objeto nomeado.

Problemático:

- Filhos herdam o file descriptor do semáforo via `fork` mas **não chamam** `sem_close(sem)` antes de `exit(0)`. POSIX especifica que cada `sem_open` deve ter um `sem_close` correspondente
no mesmo processo. Em Linux o kernel fecha no exit, mas o reference count do objeto nomeado
pode ficar incorreto se o pai fizer `sem_unlink` enquanto os filhos ainda têm o fd aberto.
- Filhos **não chamam** `shmdt(counter)` antes de `exit(0)` — inconsistência com P1 que faz corretamente. Idem: kernel limpa, mas não é prática POSIX.

Tabela-resumo de conformidade

Exp	APIs usadas corretamente	Ausente / incorreto
T1	`pthread_create`, `pthread_join`	`volatile`, data race C11
T2	lock/unlock, `INITIALIZER`, unlock no exit	`pthread_mutex_destroy`
P1	`shmget/shmat/shmdt/shmctl`, `fork/wait`, `shmdt` no filho	`volatile` (menor risco)
P2	`sem_open/wait/post/close/unlink`, `sem_post` no exit, cleanup no pai	`sem_close` no filho, `shmdt` no filho

---

Análise das execuções (hardware: Intel i5-7200U — 2 cores físicos / 4 lógicos)

T1 e P1 — Sem sync: rápidos, resultados "por sorte" no x86

T1 acertou exatamente 1 000 000 000 nos três casos. Isso não é ausência de corrida — o tráfego de coerência de cache **já está acontecendo** (evidenciado pelo tempo crescendo 3.4s → 4.9s → 6.5s). O que impede overshoots é o **modelo de memória TSO do x86**: quando uma thread
sobrescreve o incremento de outra, a perdedora re-lê o valor atualizado e continua loopando até atingir TARGET. Em ARM ou RISC-V (modelos de memória mais fracos) os resultados seriam frequentemente abaixo de 1B.

P1 mostra a corrida **real**: `+1` com N=2, `+4` com N=8. Processos não compartilham cache lines — a memória trafega via RAM e a janela de inconsistência é maior, tornando os overshoots visíveis.

T2 — Mutex: correto mas o pior caso é N=4

Com 2 cores físicos, 4 threads competindo por um único mutex é o pior cenário de contenção: cada `pthread_mutex_lock` que falha força context switch, e a cache line do mutex fica sendo transferida entre cores constantemente. N=4 (2m25s) é mais lento que N=8 (1m41s) porque com 8
threads o scheduler distribui melhor a espera, reduzindo o overhead de chaveamento relativo ao trabalho feito.

sys time de 44s (N=2) a 3m18s (N=4) confirma que a maior parte do tempo é kernel gerenciando
futexes, não trabalho de usuário.

P2 — Semáforo nomeado: serialização total, escala linearmente para pior

P2 escala quase linearmente: 2m27s → 4m55s → 8m16s. Cada incremento exige duas syscalls completas (`sem_wait` = `futex(WAIT)` + `sem_post` = `futex(WAKE)`), passando pelo namespace de
semáforos nomeados do kernel. Com N=8, sys time (14m37s) > user time (14m34s) — o sistema passa
mais tempo no kernel gerenciando o semáforo do que executando código de usuário. Não há paralelismo real: a seção crítica engloba toda a operação.

Gráfico — Tempo real por experimento (escala logarítmica)

```
╔═══════════════════════════════════════════════════════════════════════╗
║        Tempo de execução real por experimento (escala log)            ╒
║  T1=▒  T2=▓  P1=▓  P2=║                                              ╒
║═══════════════════════════════════════════════════════════════════════╒
║ T1 N=2   |▒▒▒
```